

بسمه تعالی

احسان صالحی 40327603

سوال 1:

(الف) باید از فیلتر 'not ip' استفاده کرد.

عبارت ip تمام بسته های ip را فیلتر می کند ، سپس با استفاده از عملگر not ، آنها را حذف کرده و فقط بسته های غیر ip باقی میمانند.

(ب) پس از اجرای tcpdump با فیلتر گفته شده و حذف یکی از رکورد های جدول ARP ، هنگام برقراری مجدد ارتباط با مقصد ، در خروجی بسته های مربوط به ARP مشاهده شدند. به اینصورت که سیستم ابتدا یک ARP Request ارسال کرد سپس دستگاه مقصد با یک ARP Reply آدرس MAC خود را ارسال نمود.

```
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens33, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:00:23.223353 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:00:24.563307 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:00:25.213946 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:00:26.219602 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:00:27.564359 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:00:28.217084 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:00:29.222017 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:00:30.577249 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:00:31.214380 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:00:32.219228 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:00:33.583313 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:00:34.217759 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:00:35.209562 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:02:23.155698 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:02:24.514612 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:02:25.149733 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:02:26.155610 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:02:27.523123 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:02:28.144996 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:02:29.154046 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:02:30.535378 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:02:31.154333 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:02:32.142228 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:02:33.549366 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:02:34.140974 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:02:35.153762 ARP, Request who-has _gateway tell 192.168.88.1, length 46
13:03:43.799529 ARP, Request who-has ehsan-VMware-Virtual-Platform tell _gateway, length 46
13:03:43.799540 ARP, Reply ehsan-VMware-Virtual-Platform is-at 00:0c:29:df:ef:7c (oui Unknown), length 28
```

همانطور که مشاهده میشود ، سیستم آدرس MAC مربوط به رکورد حذف شده (**192.168.88.1**) را ندارد ، به همین دلیل لایه

لینک ARP Request را در شبکه بصورت برو دکست ارسال میکند تا آن را پیدا کند. در ادامه دستگاه مقصد یا گیت وی که

صاحب IP است ، آدرس MAC رو با دستور ARP Reply ارسال میکند.

دلیل این رفتار این است که پس از حذف رکورد از جدول ARP ، سیستم دیگر آدرس MAC مربوط به IP حذف شده رو نداشت ،

بنابراین مجبور شد با استفاده از پروتکل ARP آدرس MAC مقصد را مجدداً کشف کند. به همین دلیل بسته های ARP در

خروجی tcpdump دیده شد.

```
ehsan@ehsan-VMware-Virtual-Platform:~$ sudo tcpdump -i ens33 'not ip'
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens33, link-type EN10MB (Ethernet), snapshot length 262144 bytes
12:25:51.883405 ARP, Request who-has _gateway tell 192.168.88.1, length 46
12:25:52.919457 ARP, Request who-has _gateway tell 192.168.88.1, length 46
12:25:53.887218 ARP, Request who-has _gateway tell 192.168.88.1, length 46
12:25:54.880570 ARP, Request who-has _gateway tell 192.168.88.1, length 46
12:25:55.938588 ARP, Request who-has _gateway tell 192.168.88.1, length 46
12:25:56.884928 ARP, Request who-has _gateway tell 192.168.88.1, length 46
12:25:57.884449 ARP, Request who-has _gateway tell 192.168.88.1, length 46
12:25:58.947894 ARP, Request who-has _gateway tell 192.168.88.1, length 46
12:25:59.886868 ARP, Request who-has _gateway tell 192.168.88.1, length 46
12:26:00.884261 ARP, Request who-has _gateway tell 192.168.88.1, length 46
12:26:01.963078 ARP, Request who-has _gateway tell 192.168.88.1, length 46
12:26:02.884484 ARP, Request who-has _gateway tell 192.168.88.1, length 46
12:26:03.893955 ARP, Request who-has _gateway tell 192.168.88.1, length 46
12:26:31.601612 ARP, Request who-has 192.168.88.254 tell ehsan-VMware-Virtual-Platform, length 28
12:26:31.602494 ARP, Reply 192.168.88.254 is-at 00:50:56:e1:27:ff (oui Unknown), length 46
```

همان طور که در خروجی ترمینال مشاهده می شود ، نام ماشین مجازی سیستم من ehsan-VMware-Virtual-Platform است. به دلیل پاک شدن جدول ARP ، سیستم دیگر آدرس MAC مقصد را در حافظه ندارد ، بنابراین ابتدا یک بسته ARP Request به صورت برودکست ارسال می کند تا آدرس MAC گیت وی مجازی یعنی **192.168.88.254** را پیدا کند:

12:26:31.601612 ARP,Request who-has 192.168.88.254 tell ehsan-VMware-Virtual-Platform, length 28

12:26:31.602494 ARP, Reply 192.168.88.254 is-at 00:50:56:e1:27:ff (oui Unknown), length 46
و در انتها گیت وی با یک ARP Reply آدرس MAC خواسته شده رو دراختیاری ماشین مجازی میگذارد.

از آنجا که آزمایش در محیط VMware و در حالت NAT انجام شده است ، ماشین مجازی به لایه دوم شبکه فیزیکی هات اسپات دسترسی مستقیم ندارد. سیستم عامل ویندوز به عنوان واسط بین ماشین مجازی و شبکه عمل کرده و تنها ترافیک مورد نیاز ماشین مجازی را عبور می دهد. در نتیجه ، tcpdump در ماشین مجازی عمدتاً ترافیک محلی مرتبط با خود ماشین و Gateway را مشاهده می کند و به همه بسته های لایه دوم موجود در شبکه فیزیکی دسترسی ندارد.

سوال 2:

ساختار sockaddr:

یک ساختار عمومی است برای نگهداری آدرس در سوکت‌های مختلف که بصورت زیر است:

```
struct sockaddr {  
    sa_family_t sa_family;  
    char sa_data[14];  
};
```

که در آن sa_family مشخص کننده خانواده آدرس است و sa_data هم داده مربوط به آدرس را نگهداری می کند.

ساختار sockaddr_ll:

این ساختار برای سوکت های لایه لینک استفاده میشود و آدرس های دقیق تری از کارت شبکه و آدرس های MAC در خود ذخیره می کند.

```
struct sockaddr_ll {  
    unsigned short  sll_family;  
    __be16          sll_protocol;  
    int             sll_ifindex;  
    unsigned short  sll_hatype;  
    unsigned char   sll_pkttype;  
    unsigned char   sll_halen;  
    unsigned char   sll_addr[8];  
};
```

sll_family: برای مشخص کردن خانواده آدرس است.

sll_protocol: مشخص کننده نوع پروتکل لایه لینک استفاده میشود.

sll_ifindex: شماره رابط شبکه را ذخیره می کند.

sll_hatype: نوع سخت افزار شبکه را مشخص می کند.

sll_pkttype: مشخص کننده نوع بسته است (Multicast, Broadcast,...)

sll_halen: برای ذخیره طول آدرس MAC

sll_addr: آدرس MAC

سوال 3:

از آنجایی که هدر اترنت 14 بایت است ، ساختار آن به شکل زیر است :

6 بایت اول : آدرس MAC مقصد

6 بایت دوم : آدرس MAC مبدا

2 بایت آخر : نوع پروتکل لایه بالاتر

زمانی که سیستم یک بسته را به عنوان Incoming نشان می‌دهد ، یعنی مقصد نهایی این بسته خودِ دستگاه ماست. بنابراین ۶ بایت اول این بسته ، آدرس MAC دستگاه ما را نشان می‌دهد.

با بررسی 6 بایت اول بسته Incoming که آدرس MAC همان دستگاهی است که برنامه بر روی آن اجرا شده است ، داریم :

Source MAC = ac:7b:a1:4f:4c:0f

سوال 4:

در ARP , Outgoing اول یک ARP Request داریم که درخواست آدرس MAC مربوط به آیپی **192.168.1.1** که همان آیپی گیت وی روتر است رو می‌کند (چون $c0\ a8\ 01\ 01 = 192.168.1.1$). سپس در ARP , Incoming یک ARP Reply

داریم که می‌گوید : c0a80101 647002f0f616

این بدین معناست که آدرس MAC مربوط به **IP = 192.168.1.1** برابر با 64:70:02:f0:f6:16 است.

سوال 5:

بله ، در این برنامه استفاده از تابع `ntohs()` ضروری است. زیرا پروتکل های مربوط به شبکه مطابق استاندارد خود ، داده های چند بایتی رو به صورت `Network Byte Order (Big-Endian)` ارسال می کنند. یعنی بایت با ارزش تر ابتدا فرستاده می شود. اما از طرف دیگر پردازنده های معماری `Intel x86/x64` داده ها را در حافظه به صورت

`Host Byte Order (Little-Endian)` ذخیره می کنند. یعنی بایت کم ارزش تر در آدرس ابتدایی قرار میگیرد.

در نتیجه ، تابع `() ntohs` که مخفف `Network to Host Short` است ، مقدار ۱۶ بیتی `h_proto` در ساختار

`struct ethhdr` که نوع پروتکل لایه بالاتر را مشخص می‌کند ، می‌گیرد و مقدار ۱۶ بیتی را از ترتیب بایت شبکه به ترتیب بایت میزبان تبدیل می‌کند. در این برنامه قبل از مقایسه `h_proto` با مقادیر ثابتی مانند `ETH_P_IP` ، `ETH_P_ARP` و

`ETH_P_IPV6` از `() ntohs` استفاده شده است تا پروتکل لایه بالاتر به درستی تشخیص داده شود.

سوال 6:

با توجه به فیلد Protocol در هدر IP ، پروتکل های موجود در بسته های شکل عبارتند از:

شماره پروتکل 6 مربوط به پروتکل TCP است.

شماره پروتکل 17 مربوط به پروتکل UDP است.

شماره پروتکل 2 مربوط به پروتکل IGMP است.

بنابراین طبق خروجی برنامه ، پروتکل بعد از پروتکل IP در بسته های نشان داده شده به ترتیب:

TCP و UDP و IGMP و IGMP

سوال 7:

الف) همانطور که در سوال 5 هم گفته شد ، در استاندارد شبکه (Big-Endian) ، بایت های با ارزش تر داده ها ابتدا فرستاده میشود ، در حالی که نحوه قرارگیری Bit-Fields در حافظه سیستم های Little-Endian و Big-Endian متفاوت است و در Little-Endian بایت های کم ارزش تر ابتدا فرستاده میشوند. بنابراین ، به دلیل تفاوت معماری پردازنده ها در نحوه ذخیره داده ها در حافظه و جلوگیری از جابه جا شدن دو فیلد ip_v و ip_hl و تفسیر نادرست آنها ، از این شرط ها قبل از کامپایل استفاده می شود.

پردازنده های Intel از معماری Little Endian استفاده می کنند ، بنابراین در آن ها شرط

__BYTE_ORDER == __LITTLE_ENDIAN برقرار است.

ب) 4: در کنار متغیرهای ip_v و ip_hl نشان دهنده تعریف Bit-Field در زبان C است. این عدد مشخص می کند که هر متغیر فقط 4 بیت فضا می گیرد تا این دو فیلد در مجموع یک فضای 8 بیتی (1 بیتی) اشغال کرده باشند.

اگر ساختار گفته شده تنها شامل تعریف همین دو متغیر بود ، در اینصورت خروجی دستور sizeof(struct ip) برابر با 4 بایت می شد.

چرا؟

اگرچه مجموع بیت های مورد نیاز این دو متغیر 8 بیت (1 بایت) است ، اما هر دو Bit-Field در کد ، درون یک متغیر unsigned int تعریف شده اند. طبق قواعد زبان C ، کامپایلر Bit-Field ها را بر مبنای اندازه نوع متغیر آنها فضا میدهد. و چون unsigned int برابر با 4 بایت است ، خروجی sizeof برابر با 4 خواهد بود.